

Radiology STT Pipeline

Implementation Plan, Strategies, and System Configuration

Implementation Strategy

The successful deployment of a real-time, highly accurate medical dictation system requires a robust architecture that prioritizes patient data security, minimal latency, and high transcription accuracy. The strategies below dictate the foundational approaches for this dual-model (Gemini & GPT) pipeline:

- **Security Isolation & HIPAA Compliance Preparation:** The frontend application must act solely as an audio capture and rendering layer. All AI model API keys (Gemini, OpenAI) are kept strictly on the Python backend. No raw text or audio should be logged permanently without encryption, ensuring a pathway to HIPAA compliance.
- **Bandwidth & Payload Optimization:** Real-time streaming requires efficient data transfer. The raw microphone input (typically 44.1kHz or 48kHz Float32) is downsampled to 16kHz Int16 PCM directly on the client-side. 16kHz is the optimal frequency for capturing human speech (85-255Hz) while keeping the WebSocket payloads small, enabling sub-500ms latency.
- **Noise Gate Control via VAD:** Radiology reading rooms often have background noise (typing, other dictations). A WebRTC Voice Activity Detection (VAD) layer on the Python backend serves as a noise gate. It analyzes 10ms/20ms audio frames and filters out silence and background hums, preventing the transmission of empty or noisy chunks to the AI models, which reduces hallucinated transcripts and saves API costs.
- **Bidirectional Streaming Architecture:** Asynchronous WebSockets are utilized across the entire pipeline. The React frontend streams continuous audio to the backend, which simultaneously streams it to the AI providers. Text chunks are immediately streamed back to the frontend as they are generated, providing a seamless "live typing" experience for the radiologist.

Phased Implementation Plan

Phase 1: Architecture & Infrastructure Setup

- Initialize the Python backend using FastAPI and Uvicorn for asynchronous server capabilities.
- Set up the React frontend (Vite or Create React App) with standard functional components.
- Install necessary backend dependencies: `webRTCvad`, `google-genai`, `openai`, and `websockets`.
- Configure environment variables for secure API key storage.

Phase 2: Frontend Audio Pipeline Construction

- Implement the Web Audio API in the React client to capture raw microphone input.
- Build a custom `ScriptProcessor` or `AudioWorklet` to convert Float32 audio to Int16 PCM (Little Endian).
- Chunk the processed audio into exact 320-byte buffers (representing 10ms frames) to align with WebRTC VAD requirements.
- Establish the WebSocket connection to transmit these chunks to the backend.

Phase 3: Backend Routing & Filtering

- Build FastAPI WebSocket endpoints specifically structured to receive continuous binary audio frames.
- Integrate `webRTCvad` to evaluate incoming frames. Route only frames containing active human speech.
- Develop the dual-routing logic to branch the stream to either the Gemini Live API or the OpenAI Realtime API based on the client's request.

Phase 4: AI Model Configuration & Refinement

- Inject the strict radiology system instructions into the session configuration of the chosen AI model.
- Force the models to output only text modalities (disabling audio generation).
- Implement error handling for WebSocket disconnects and timeout reconnections.

- Test with complex medical terminology (e.g., Concha bullosa, mucosal thickening) to verify transcription accuracy.

Radiology System Prompt

This prompt must be injected into the system instructions for both the Gemini and GPT initialization calls to ensure the AI behaves strictly as a medical STT engine.

SYSTEM INSTRUCTION

ROLE: You are an expert real-time medical transcription engine specializing in radiology dictation. TASK: Transcribe the user's speech verbatim into highly accurate medical text. CONSTRAINTS: 1. Do not engage in conversation, answer questions, or provide pleasantries. Output ONLY the transcribed text. 2. Intercept and convert spoken punctuation commands into their respective symbols (e.g., 'period' -> '.', 'comma' -> ',', 'next line' -> ' ', 'new paragraph' -> ' '). 3. Ensure flawless spelling for complex anatomical terms, pathologies, and medical phrasing (e.g., Concha bullosa, bilateral inferior turbinates, hypertrophy). 4. Maintain proper capitalization for the start of new sentences and specific anatomical acronyms.